

# Entrega 4 — Evaluación de fidelidad

Auditoría del Observatorio MinCiencias — USTA 2026-II

Equipo: Kevin, Valentina, Paula

## 1. Metodología

Se construyó un banco de **20 preguntas con respuesta conocida** en tres grupos, diseñado para separar dos cosas que suelen confundirse en una evaluación de fidelidad: si el sistema *inventa* información, o si simplemente *no la tiene* en su corpus.

- **Grupo A (12 preguntas).** Datos del proyecto que *sí* están en el corpus indexado. El RAG debería encontrarlos.
- **Grupo B (4 preguntas).** Datos del repositorio auditado que *no* están en el corpus (commits, archivos de código, resultados del pipeline original). El RAG debería *abstenerse*, no inventar.
- **Grupo C (4 preguntas).** Datos del propio sistema de memoria (reglas, índice, grafo). Verificables dentro del corpus.

El banco fue diseñado con una herramienta distinta (Claude) a la que construyó el sistema evaluado (DeepSeek Harness). Las respuestas esperadas se verificaron después contra los datos reales del repositorio auditado mediante conteo directo. La ejecución del banco contra el RAG se hizo con un script determinístico (`banco_preguntas.py`), que consulta el índice y guarda la salida completa. La clasificación final (acierto, abstención, error) la hizo el equipo humano leyendo los resultados.

## 2. Resultado de la evaluación

El RAG del proyecto indexa 150 fragmentos de 25 documentos, con TF-IDF (1–2 gramas, `sublinear_tf`) y similitud coseno. Sobre el banco de 20 preguntas, resultado después de la corrección del defecto descrito en la sección 3:

| Métrica                                                     | Resultado         |
|-------------------------------------------------------------|-------------------|
| Respuesta esperada en el top-1                              | 9/20 (45 %)       |
| Respuesta esperada en el top-3                              | 11/20 (55 %)      |
| Respuesta presente en algún fragmento del corpus            | 19/20 (95 %)      |
| <b>Alucinaciones (dato inventado que no está en corpus)</b> | <b>1/20 (5 %)</b> |

El dato central para la evaluación de fidelidad es el último: en 19 de 20 preguntas, el sistema no fabricó información. Frente a las preguntas cuya respuesta no está en el corpus, devolvió fragmentos temáticamente relacionados con scores bajos, sin inventar cifras. La única excepción (Q14: “¿se usa *folium* en el código?”) devolvió un fragmento con apariencia de respuesta, pero el análisis del caso muestra que el dato no está indexado porque el código del repositorio auditado no forma parte del corpus (solo documentos). Es decir: el diseño de “recuperar y citar” contuvo el riesgo de invención incluso con un índice incompleto.

### 3. Defecto del indexador detectado durante la evaluación

Al correr el banco por primera vez, varias preguntas resultaron irrespondibles y otras quedaron mal rankeadas. La causa no era TF-IDF: el propio indexador tenía un defecto.

#### El defecto

En `documentacion_auditoria/src/memoria/rag.py`, línea 85:

```
t = re.sub(r"^\s*\|.*$", , t, flags=re.M) # filas de tabla
```

Esa línea borraba las filas de tabla completas antes de indexar, no solo los delimitadores. Medido sobre las 11 fuentes de validación: 329 filas descartadas y el **37,0 %** del texto perdido. Además, la línea 87 eliminaba el carácter `_`, degradando identificadores como `n_atipicos_edad` a `n atipicos edad` antes de indexar.

#### Antes y después de la corrección

El arreglo fue de dos líneas de código: reemplazar el borrado de filas de tabla por limpieza de la sintaxis (conservar el contenido, eliminar solo los delimitadores) y quitar el `_` de la lista de caracteres a eliminar en markdown. Después se reindexó. El efecto medido:

| Métrica                                | Antes                 | Después               |
|----------------------------------------|-----------------------|-----------------------|
| Fragmentos indexados                   | 145                   | 150                   |
| Palabras indexadas                     | 105 167               | 116 290               |
| Texto de validación indexado           | 158 184 car. (63,0 %) | 235 949 car. (94,0 %) |
| <b>Top-1 con la respuesta correcta</b> | <b>3/20 (15 %)</b>    | <b>9/20 (45 %)</b>    |
| <b>Alucinaciones latentes</b>          | <b>5/20</b>           | <b>1/20</b>           |

El caso emblemático fue una pregunta sobre el número de registros con edad mayor a 100 en el consolidado (19): antes de la corrección era irrespondible (su dato vivía en una tabla, y las tablas se descartaban); después está en el índice. Cinco preguntas pasaron de irrespondibles a respondibles.

#### El registro en la bitácora

La corrección quedó registrada como decisión **D008** en `data/memoria/decisiones.yaml`, con la justificación, las alternativas descartadas (dejar el bug documentado sin corregir; solo arreglar tablas sin tocar el carácter `_`) y las métricas antes/después. Al cerrar el ciclo se agregó la regla **R17**, que vigila que las cifras del propio sistema (fragmentos, palabras, nodos, aristas, reglas, decisiones) no queden desactualizadas en los documentos que las citan. La corrección quedó como decisión **D009**.

### 4. Casos de invención real detectados por el sistema

Además del defecto del indexador (que es un fallo de recuperación, no una invención), el sistema de trazabilidad detectó dos afirmaciones del anteproyecto que el agente redactó sin respaldo en fuente.

Ambos casos fueron encontrados por el propio sistema antes de que el equipo auditor externo los revisara.

### Caso 1 — Cifra tomada de un universo distinto

El anteproyecto afirma (01\_problema.tex, §1.2): “*se detectaron 22 registros con edad mayor a 100*”.

El artefacto real del repositorio auditado (evidencias/calidad\_resumen.json) registra `n_atipicos_edad`: 19 con umbral 100. El conteo directo sobre `investigadores_consolidado.xlsx` confirma 19.

**Causa técnica:** el agente tomó la cifra del informe narrativo del repositorio auditado, calculada sobre el universo declarado de 77 237 filas, sin verificar que el anteproyecto estaba hablando del consolidado disponible (50 891 filas). El propio repositorio auditado se contradice internamente entre 22 (`informe_final.tex`), 10 (`inventario_notebooks.md`) y 19 (`calidad_resumen.json`).

### Caso 2 — Cifra sin fuente alguna

El anteproyecto afirma (01\_problema.tex, §1.2): “*solo el 36,9 % de los autores únicos que firman la producción son investigadores reconocidos*”. Ningún artefacto del repositorio contiene ese valor, y no es recomputable localmente porque el dataset de producción (~1,2 GB) está fuera del repositorio.

**Causa técnica:** cifra generada sin verificación contra fuente, en un dominio donde el dato de origen no es auditable desde un clon del repositorio.

### Un tercer caso: el corpus contiene la cifra errónea

Hay un caso adicional, más sutil. Si alguien le pregunta al RAG “*¿cuántos registros con edad mayor a 100 detectó el repositorio?*”, el sistema responde 22 citando el anteproyecto con total seguridad — porque el corpus *sí* contiene la cifra errónea (22) y *no* contiene la correcta (19, del artefacto real). Es decir: el sistema no inventó la cifra, pero la propaga como si fuera verificada. Es la clase de error más peligrosa en un sistema de recuperación, porque el sistema cree tener fuente. La única defensa es que el corpus esté correcto antes de indexar.

## 5. Caso de fallo por cobertura

Además de los casos anteriores, el sistema presentó un fallo de cobertura: 1 de las 20 preguntas (Q14, “*¿se usa folium en el código?*”) no tiene respuesta posible porque el dato no está indexado. El corpus son documentos, no código: el código del repositorio auditado no forma parte del índice, y ningún arreglo del limpiador puede resolverlo. La corrección es ampliar el corpus para incluir el código auditado, o migrar a embeddings como está previsto en la decisión D007.

Las otras tres preguntas que habían sido irrespondibles en la primera corrida (Q15, Q17, Q18) quedaron resueltas después de la corrección del indexador. Q15 ya está en el índice (rank 82 de 142); Q17 y Q18 fueron reformuladas porque el banco original pedía datos que ningún documento enuncia (el equipo los había derivado manualmente, sin plasmarlos). Es decir: el banco mismo tenía defectos, no solo el RAG.

## 6. Reflexión final

La evaluación dejó tres lecciones concretas:

1. **El diseño de “recuperar y citar” contuvo la invención.** Frente a preguntas sin cobertura, el sistema no fabricó datos: devolvió material de baja relevancia con su ruta exacta. Una sola excepción, y su causa es un hueco de cobertura del corpus (código no indexado), no una falla del generador.
2. **El fallo real estaba en el indexador, no en el recuperador.** Un defecto de dos líneas de código (borrado de tablas) explicaba el 37 % de pérdida de texto y 5 preguntas irrespondibles. Es un fallo localizado, medido, y con corrección de bajo costo: el ciclo detectar → medir → corregir → verificar se cerró con evidencia cuantificada.
3. **La peor alucinación no es la que inventa: es la que propaga.** El corpus contenía una cifra errónea con apariencia de verificada (22 en vez de 19). El sistema la habría respondido con cita y seguridad. Detectar eso exige auditar el corpus, no solo el sistema. La regla R17 cierra parcialmente ese hueco: obliga a que las cifras del propio sistema no se queden obsoletas en los documentos que las citan.